Stock Market Index Direction Prediction Using Limit Order Book Data

MTech (DSAI) Capstone — Final Revision

Joseph Raj Sumanth G.M Abhishek Kumar Kumara Swamy Raghav Saagar

June 2026

Abstract

We study short-term *index-direction* prediction from Limit Order Book (LOB) data, combining a faithful reproduction of three published deep LOB models (DeepLOB, MLPLOB, TLOB) on the public FI-2010 benchmark with a first-of-its-kind transfer study on **NSE index futures (NIFTY / BANKNIFTY)**, for which we *engineered our own dataset*. We additionally introduce **MambaLOB**, a selective state-space (SSM) architecture, and characterise its efficiency–accuracy trade-off against the transformer baseline. A substantial part of the contribution is **data engineering**: because no clean Indian index-futures LOB dataset existed, we reverse-engineered the Dhan binary depth feed, built a cloud collection pipeline (EC2 $\rightarrow$ S3), and produced a continuous front-month series with a documented preprocessing pipeline that aligns the NSE data to the FI-2010 contract. This document is the final interim revision: it reports the reproduction, the engineered NSE benchmark, the architecture comparison, and an efficiency analysis, with honest framing of what does and does not transfer.

Contents

1 Introduction and Literature Survey

1.1 Background and Context

Financial markets facilitate capital allocation, risk distribution, and continuous price discovery. Early forecasting relied on statistical time-series models applied to closing prices. Modern electronic exchanges instead operate as continuous double auctions recorded in a *Limit Order Book* (LOB), which updates at microsecond granularity and produces millions of structured events per session. For an index such as **NIFTY 50** — India's premier equity index — the LOB of the index-*futures* contract encapsulates the collective order flow of a broad basket of large-cap stocks, offering a rich microstructural signal for short-horizon direction prediction.

1.2 Motivation and Importance of the Study

Research on deep LOB models is concentrated on developed markets (NASDAQ, Nordic exchanges). Indian index futures are highly liquid yet structurally distinct (tick discreteness, regime shifts, distinct liquidity profiles), and remain almost entirely unstudied with modern LOB architectures. Index direction is hard to predict because the series is non-stationary, exhibits stochastic volatility, and is governed by rapidly evolving supply–demand dynamics. Crucially, the LOB evolves in *event time*, not clock time: it updates whenever a limit order is placed, a market order executes, an order is cancelled, or a level is modified. Most prior deep models resample this irregular stream onto a fixed grid (e.g. 1-second snapshots), which loses intra-interval dynamics, blurs microstructure reactions, and introduces sampling bias. This study targets Indian index futures directly in event time, addressing a clear geographic and methodological gap.

1.3 Problem Definition and Scope

We predict the direction of the future mid-price over a short horizon from a window of recent LOB states. The scope comprises: (i) high-frequency LOB inputs (top-k bid/ask prices and volumes); (ii) construction of an event-aligned NSE index-futures dataset; (iii) reproduction of CNN, MLP, and Transformer baselines; (iv) a novel selective-SSM architecture (MambaLOB); and (v) multi-horizon, cost-aware evaluation.

1.4 Research Objectives and Questions

Objectives.

- Reproduce DeepLOB and TLOB on FI-2010 and validate against published numbers.
- Engineer a reproducible, event-aligned NSE index-futures LOB dataset (NIFTY, BANKNIFTY).
- Introduce MambaLOB and characterise its accuracy and computational efficiency.
- Compare all architectures on identical data, labels, and protocol; quantify transfer to NSE.

Research Questions.

1. Do our reproductions match published FI-2010 results within tolerance?
2. Is a linear-time selective SSM (Mamba) competitive with a transformer at lower compute?
3. Does LOB directional signal transfer from FI-2010 to NSE index futures, and how much does it degrade?
4. Which labelling scheme (fixed-threshold vs spread-relative) yields an economically meaningful task?

1.5 Expected Contributions

- **Engineered NSE benchmark.** A continuous front-month NIFTY/BANKNIFTY index-futures LOB dataset built from the Dhan depth feed — to our knowledge the first such Indian index-futures LOB dataset prepared for ML, including the full data-engineering pipeline.
- **Reproduction.** Faithful re-implementations of DeepLOB, MLPLOB and TLOB with results benchmarked against the original papers and against independent reproductions (LOBCAST).
- **Novel architecture.** MambaLOB, a selective state-space model for LOB, with a rigorous efficiency comparison (linear vs quadratic scaling in sequence length).
- **Transfer study.** A standardized comparison of four architectures on the NSE benchmark, with honest reporting of degradation relative to FI-2010.

1.6 Review Methodology and Source Selection Criteria

The survey covers classical statistical approaches (ARIMA), Hidden Markov Models, deep LOB models (CNN, LSTM, CNN–LSTM hybrids), the transformer-based TLOB/MLPLOB family, state-space models for sequences (S4/S5/Mamba), and continuous-time microstructure models. Selection criteria: use of high-frequency LOB data, explicit modelling of LOB structure, empirical validation, and architectural novelty. Primary references include the DeepLOB family, the TLOB paper, FI-2010 (Ntakaris et al.), the LOBCAST benchmark study (Prata et al.), the Mamba paper (Gu & Dao), and continuous-time LOB modelling work.

1.7 Survey of Existing Approaches

1.7.1 Early Solutions and Classical Techniques

Parametric statistical models (AR, MA, ARMA, ARIMA) assume specific data-generating processes and perform adequately on linear dynamics but fail under the non-linear, non-stationary behaviour of LOB data. Enhanced and hybrid HMM/ARIMA models improve regime modelling but depend on historical price series, do not use LOB depth, and are sensitive to regime misclassification.

1.7.2 Modern Approaches and Evolving Trends

DeepLOB (Zhang et al., 2019) combines convolutional feature extraction across LOB levels with an LSTM over time, substantially improving mid-price movement prediction on FI-2010. It established the 40-feature $\times$ window representation that we adopt. It nonetheless operates on event-resolution snapshots without attention.

1.7.3 Emerging Models and Hybrid Frameworks

MLPLOB and **TLOB** (Berti & Kasneci, 2025) reframe the model as alternating normalization and mixing blocks; TLOB uses *dual* (temporal and feature) self-attention and is the current state-of-the-art on FI-2010. **State-space models** (S4, S5, Mamba) offer linear-time sequence modelling and have entered LOB research on the generative side; their use as a classification backbone for LOB direction is the novelty window this project targets.

1.8 Comparative Evaluation of Techniques

1.9 Dataset and Benchmark Overview

FI-2010 (public benchmark): 5 Finnish stocks, 10 trading days, 10-level LOB, pre-normalized (z-score variant), with standard anchored cross-folds. Used for reproduction. **NSE index-**

Table 1: Comparison of LOB modelling approaches considered in this work.

Approach	Strength	Limitation	Role here
ARIMA	Linear forecasting	No non-linear/LOB structure	Conceptual baseline
HMM	Regime modelling	No microstructure depth	Conceptual baseline
DeepLOB (CNN–LSTM)	Depth + temporal	Sequential bottleneck	Reproduced baseline
MLPLOB	Lightweight	No sequence model	Ablation floor
TLOB (Transformer)	SOTA accuracy	$O(L^2)$ attention cost	Reproduced SOTA baseline
MambaLOB (SSM)	$O(L)$, tiny params	Lower accuracy on FI-2010	**Proposed model**

futures dataset (engineered here): NIFTY and BANKNIFTY near-month futures, 20-level depth captured live via the Dhan API and stored as Parquet on S3; the first ten levels yield a 40-feature vector identical in layout to FI-2010. Period: 12 May – 12 June 2026 (21 valid trading days across the May → June expiry roll); ≈57k events/instrument/day.

1.10 Tools, Frameworks, and Experimental Protocols

Python 3.x; PyTorch (≥2.x) with the CUDA `mamba-ssm` kernel; scikit-learn (metrics, baselines); NumPy/Pandas (LOB processing); AWS EC2 + S3 (collection/storage); Google Colab and Kaggle (GPU training); Matplotlib/Seaborn (figures). Protocol: chronological train/val/test splits; per-feature z-score from training statistics only; sliding windows of length $L = 100$; percentage-mid-change labelling.

1.11 Evaluation Metrics Used in Literature

Table 2: Evaluation metrics for LOB direction prediction.

Metric	Definition	Why it matters
Weighted F1	class-frequency-weighted F_1	matches headline numbers in original papers
Macro F1	unweighted mean F_1	fair under class imbalance; matches independent
Accuracy	correct/total	supplementary; misleading under imbalance
MCC	Matthews correlation	balanced single-number summary
Hit-rate / net bps	directional accuracy / cost-adj. return	signal-quality / economic relevance

We report both weighted and macro F_1 throughout, because (as Section 3.6 shows) the original FI-2010 headline numbers are weighted while independent reproductions report macro.

1.12 Limitations of Prior Work

No emerging-market (Indian) application; limited event-driven deep architectures; little use of linear-time state-space models for LOB classification; limited reporting of efficiency/scaling; and inconsistent metric conventions across papers.

1.13 Research Gaps and Open Questions

Can LOB-based models generalise to Indian index futures? Can a linear-time SSM match a transformer at lower compute? How should labels be defined so that the task is economically meaningful (rather than trivially dominated by "no move")? These motivate our design.

1.14 Innovation and Conceptual Synthesis

This work synthesises classical benchmarking, CNN spatial extraction (DeepLOB), transformer dual-attention (TLOB), and linear-time state-space modelling (Mamba) on a newly engineered Indian benchmark, under one standardized protocol.

1.15 Visualizations and Knowledge Summaries

The report includes comparative tables, architecture descriptions, a data-pipeline schematic, F1-by-horizon charts, confusion matrices, and the memory/latency-vs-sequence-length scaling curve.

1.16 Academic Integrity and Citation Style

Empirical findings are attributed to their source works; conceptual explanations are original synthesis. Generative-AI assistance (Section 3.11.1) was used for drafting and is disclosed.

1.17 Summary of the Chapter

We reviewed classical, deep, transformer, and state-space approaches, identified the Indian-market and efficiency gaps, and positioned our contribution: an engineered NSE LOB benchmark, a reproduction of three baselines, and a novel Mamba-based model evaluated on identical footing.

1.18 Structure of the Report

Chapter 1 (this chapter) introduces the problem and surveys the literature. Chapter 2 establishes the theoretical and architectural foundations. Chapter 3 is the interim report: the engineered dataset, the data-engineering pipeline, preprocessing, baseline development, preliminary results, efficiency analysis, reproducibility, risks, and next steps.

2 Foundations

2.1 Introduction to the Foundations Chapter

This chapter establishes the theoretical underpinnings (market microstructure, the limits of market efficiency), the conceptual data-to-model pipeline, the four model architectures (three reproduced baselines and the proposed MambaLOB), and the notation used throughout.

2.2 Theoretical Underpinnings

2.2.1 Market Microstructure

The continuous double auction maintains a LOB recording all outstanding limit orders at each price level. The *bid* side (buy orders, descending) has best bid P_1^b; the *ask* side (sell orders, ascending) has best ask P_1^a. The mid-price is the primary, less noisy, forecasting target:

$$p_{\text{mid}}(t) = \frac{P_1^a(t) + P_1^b(t)}{2}. \tag{1}$$

The bid–ask spread $S(t) = P_1^a(t) - P_1^b(t)$ measures liquidity (and, importantly for us, transaction cost), while per-level depth encodes short-term price pressure.

2.2.2 The Efficient Market Hypothesis and Its Limits

The semi-strong EMH implies no excess return from public information, yet high-frequency studies document short-lived predictability from order-flow imbalance, inventory effects, and latency arbitrage at sub-second to minute horizons. Deep models trained on LOB snapshots exploit precisely these transient inefficiencies; their documented (if small) predictive power motivates this study, while the EMH frames our expectation that signal is weak and easily overwhelmed by costs.

2.3 Conceptual Framework

The pipeline is: *collect* (FI-2010 download; live Dhan capture for NSE) → *preprocess* (clean, reorder to a 40-feature LOB vector, z-score on train statistics, label, window) → *model* (DeepLOB, MLPLOB, TLOB baselines; MambaLOB proposed) → *evaluate* (multi-horizon classification metrics, efficiency, cost-aware backtest). Every model consumes the same tensor $X \in \mathbb{R}^{B\times L\times F}$ ($L = 100$, $F = 40$) and emits logits over three classes.

2.3.1 Feature Representation

Each LOB state is the first ten levels as $[\,P_i^a, V_i^a, P_i^b, V_i^b\,]_{i=1}^{10}$ (40 features), the FI-2010 convention. Mid-price and spread are derived from level 1 for labelling and cost accounting.

2.3.2 Baseline Models

DeepLOB (CNN + Inception + LSTM); **MLPLOB** (flatten + MLP + Bilinear Normalization, the ablation floor); **TLOB** (dual temporal/feature self-attention, SOTA). All are reproduced faithfully.

2.3.3 Proposed MambaLOB Model

MambaLOB projects the 40-feature input to d_{model}, applies a stack of selective state-space (Mamba) blocks for $O(L)$ temporal mixing, optionally a feature-axis attention, then Bilinear Normalization and a linear classifier:

$$X \xrightarrow{\text{Linear}} \text{Mamba} \times n \xrightarrow{\text{(opt. feature attn)}} \text{BiN} \xrightarrow{\text{FC}} \text{logits} \in \mathbb{R}^3.$$

On GPU it uses the CUDA `mamba-ssm` kernel; a pure-PyTorch fallback exists for CPU/debug. Phase-2 variants (a convolutional front-end and a bidirectional scan) build on this core and are detailed with the experiments (Section 3.6, Figure 9).

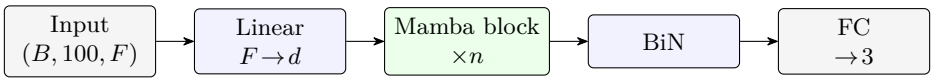

Figure 1: Proposed MambaLOB. The LOB window is projected to d_{model}, passed through n selective state-space (Mamba) blocks for $O(L)$ temporal mixing (block internals in Figure 3), then Bilinear Normalization and a 3-class head.

2.3.4 Training and Evaluation

Supervised three-class classification; class-weighted cross-entropy; AdamW with cosine decay; early stopping on validation macro/weighted-F_1; metrics as in Table 2.

Table 3: Notation.

Symbol	Meaning
$X \in \mathbb{R}^{B \times L \times F}$	input window batch ($L = 100$, $F = 40$)
L, F, B	sequence length, features, batch size
$d_{\text{model}}, d_{\text{state}}, d_{\text{conv}}$	Mamba projection / state / conv width
k (or H)	prediction horizon in LOB events
p_{mid}, S	mid-price, bid–ask spread
θ, α	label thresholds (spread-relative / fixed)

2.4 Terminology and Notational Conventions

2.4.1 Class Labels

We use the encoding $\{0 = \text{Down}, 1 = \text{Stationary}, 2 = \text{Up}\}$. Labels are assigned by the smoothed-mid percentage change over the horizon (Section 2.6).

2.5 Algorithmic Foundations

2.5.1 Data Preprocessing Algorithms

Z-score normalization per feature, using *training* statistics only: $Z = (X - \mu)/\sigma$. **Sliding-window** segmentation produces $X \in \mathbb{R}^{L \times F}$ with $L = 100$. **Labelling**: with $m^-(t)$ and $m^+(t)$ the mean mid over the k events behind/ahead, $r(t) = (m^+ - m^-)/m^-$; then Up if $r > \tau$, Down if $r < -\tau$, else Stationary, where τ is the threshold (Scheme A: fixed α; Scheme B: spread-relative θ).

2.5.2 DeepLOB / MLPLOB / TLOB

DeepLOB stacks 1×2 convolutions (fuse price/volume, then sides), a 1×10 convolution (fuse levels), an Inception module, and an LSTM. MLPLOB flattens and applies MLP blocks with Bilinear Normalization. TLOB alternates temporal and feature self-attention with a progressive MLP head. Figure 2 shows the three reproduced baselines as block pipelines.

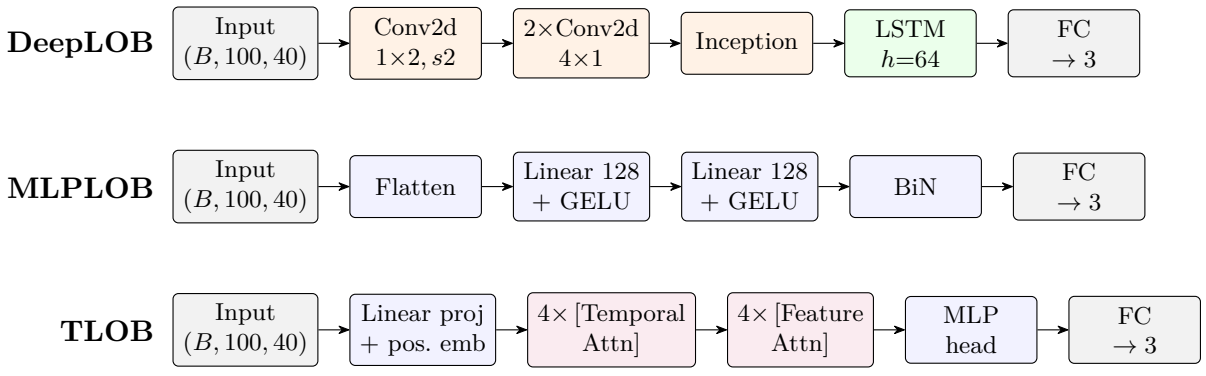

Figure 2: Reproduced baseline architectures. DeepLOB (CNN + Inception + LSTM), MLPLOB (flatten + MLP with Bilinear Normalization, the ablation floor), and TLOB (alternating temporal and feature self-attention, the transformer SOTA). All share the $(B, 100, 40) \to 3$ contract.

2.5.3 Bilinear Normalization and Attention

Bilinear Normalization (BiN) normalises across both the temporal and feature axes and learns to weight the two, stabilising LOB training. Multi-head self-attention provides cross-position

mixing in TLOB; its cost is quadratic in L.

2.5.4 Selective State-Space Modelling

A Mamba block realises a discretised, input-dependent linear recurrence $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$, $y_t = C_t h_t$, where the selection mechanism makes $\bar{A}, \bar{B}, C$ depend on the input. This yields $O(L)$ time and memory and a parameter count independent of L — the basis of our efficiency claim. Figure 3 details the block internals.

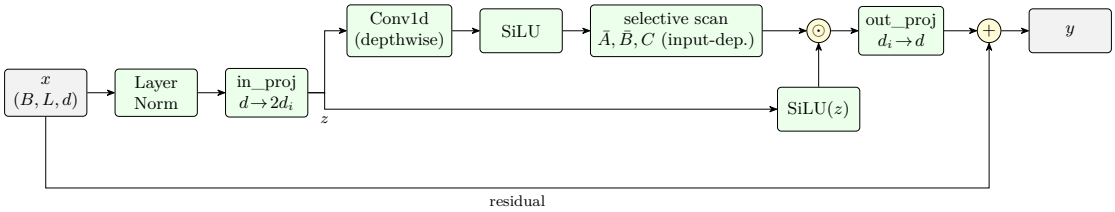

Figure 3: Selective state-space (Mamba) block. The input is normalised and projected to two streams; one passes a depthwise causal convolution and SiLU into the selective scan (with input-dependent $\bar{A}, \bar{B}, C$), the other forms the SiLU gate. The gated output is projected back and added to the residual. The scan is the only sequential step and runs in $O(L)$.

2.6 Classification Algorithm

The head produces $\hat{y} = \text{Softmax}(Wx + b)$ over the three classes. Optimization uses class-weighted cross-entropy and AdamW; evaluation uses weighted/macro F_1, accuracy, and MCC.

2.7 Model Design Principles

Modularity (separate ingest / dataset / model / train / eval modules), reusability (shared loaders, normalization, windowing across FI-2010 and NSE), adaptability (one data contract for all models), and scalability (lazy windowing keeps memory $O(T \cdot F)$ rather than materialising all windows).

2.8 Domain-specific Constraints and Assumptions

Tick discreteness (NIFTY futures move in $\geq$0.05 increments) quantizes mid-price changes and informs threshold choice; the bid–ask spread sets a hard floor on tradeable signal; pre-open and the opening/closing auction windows are excluded to isolate continuous trading; expiry days require contract rolling.

2.9 Ethical, Legal, and Societal Implications

The work uses public benchmark data and self-collected public market data (no personal data). Predictions are framed as *signal-quality* research, not trading advice; the cost-aware analysis (Section 3.10) explicitly guards against over-claiming profitability.

2.10 Summary of the Chapter

We established microstructure foundations, the shared data-to-model pipeline, the four architectures, and the notation. The next chapter reports the engineered dataset and the empirical results.

3 Interim Report

3.1 Chapter Overview

This chapter documents the work completed to date: (i) the **data-engineering** effort to build an NSE index-futures LOB dataset; (ii) the preprocessing pipeline aligning it to FI-2010; (iii) reproduction of the three baselines and the proposed MambaLOB; (iv) preliminary results on FI-2010 and the NSE transfer; (v) an efficiency analysis; and (vi) reproducibility, risks, and next steps. Experiments that are still running or deferred are marked accordingly.

3.2 Dataset Acquisition and Preprocessing

3.2.1 Dataset Description

FI-2010 (reproduction). The public NoAuction z-score variant: 5 stocks, 10 days, 10 levels, pre-normalized, with standard cross-folds (we use CF_7: 7 train + 3 test days).

NSE index futures (engineered). We built a live dataset for NIFTY and BANKNIFTY near-month futures using the Dhan twenty-level depth feed, stored as daily Parquet on Amazon S3. Key facts: 21 valid trading days (12 May – 12 June 2026), spanning the May→June expiry roll; ≈57,000 events/instrument/day (≈1.1M combined per instrument, exceeding FI-2010's ≈400k scale); the first 10 of 20 levels give a 40-feature vector identical in layout to FI-2010, so models transfer with no architectural change.

3.2.2 Data Quality Checks

We evaluated two Indian data sources before committing. Table 4 summarises the comparison that led us to choose Dhan. Dhan wins on depth, event-driven granularity (structurally consistent with FI-2010), completeness, and cost. Beyond source selection, per-day cleaning drops: rows with crossed quotes ($P_1^b \geq P_1^a$); rows with any missing level-1–10 field; and tick-level glitches (mid-price jumps >0.5% relative to the previous valid mid). Empty/partial collection days (the broken expiry-roll files of 27–29 May) are excluded; the partial 11 May session is dropped.

Table 4: Dhan vs Kite vs FI-2010 (15-min simultaneous capture).

Property	Dhan (20-depth)	Kite Connect	FI-2010
LOB levels	20	5	10
Features (10-level)	40 (direct match)	20 (needs change)	40
Update mechanism	event-driven (gap CV 3.3)	periodic ∼1s (CV 1.3)	event-driven
Median inter-tick gap	0.20 s	1.0 s	<0.1 s
L1–L10 completeness	100%	n/a	100%
Order-count field	yes	no	no
Cost	low	higher	free

3.2.3 Exploratory Data Analysis (EDA)

Active instruments reach ≈200–250 events/min in single-instrument capture; the heavy-tailed inter-tick gap distribution (coefficient of variation ≈3.3) confirms genuine *event-driven* updates rather than periodic polling. Median spreads are a few basis points (NIFTY ≈2.5 bps), which — as Section 3.6 shows — has direct consequences for labelling. Class balance varies sharply with horizon and labelling scheme (Tables in Section 3.6).

3.2.4 Pre-processing Pipelines

The pipeline that aligns Dhan to the FI-2010 contract (implemented in `modeling/nse_dataset.py`):

1. **Front-month stitching**: match each daily file by root (NIFTY/BANKNIFTY), concatenating the May and June contracts into one continuous front-month series.
2. **Feature reorder**: map Dhan's column ranges to the interleaved FI-2010 layout $[P_i^a, V_i^a, P_i^b, V_i^b] \times 10$.
3. **Cleaning**: crossed-quote, missing-level, and tick-spike filters (above).
4. **Per-day segmentation**: labels and windows are computed *within* a trading day, so the overnight gap and the expiry roll are segment boundaries — no spurious cross-day returns and no window straddles two contracts.
5. **Z-score** using training statistics only.
6. **Labelling** (Scheme A fixed-α or Scheme B spread-relative).
7. **Lazy sliding windows**: the (T, F) matrix is stored once and windows are sliced on demand, keeping memory $O(T \cdot F)$; the eager (N, L, F) expansion would exhaust memory on the full series.

Default split (chronological): train 12 May – 4 June (15 days), validation = last 10% of training windows, test 5/8/9/10/11/12 June (6 most-recent out-of-sample days). Figure 4 shows the end-to-end data-engineering flow from the live broker feed to model-ready tensors.

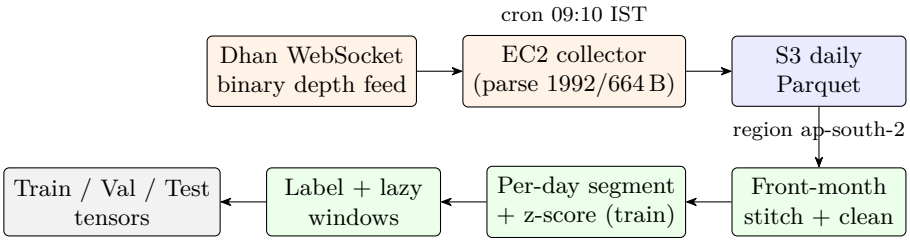

Figure 4: Engineered NSE data pipeline. The reverse-engineered Dhan binary feed (two frame formats, 1992 B full / 664 B compact) is parsed on an EC2 collector and written as daily Parquet to S3; the modelling layer then stitches the front-month series across the expiry roll, cleans, segments per trading day, z-scores using training statistics only, labels, and emits lazily-windowed tensors aligned to the FI-2010 layout.

3.3 Feature Engineering

3.3.1 Feature Identification Techniques

The base model input is the raw 40-dimensional LOB vector (10 levels) per event. On top of this we engineer *microstructure* channels (`modeling/features.py`), motivated by the weak raw-LOB signal on NSE:

- **Order-Flow Imbalance (OFI)** (Cont et al.) at level 1 and summed over levels 1–5, computed per trading day so flow never crosses an overnight/expiry boundary;
- **depth imbalance** (level 1 and aggregate over 10 levels), **micro-price** (Stoikov, volume-weighted mid), and **relative spread**;
- **per-level order counts** — a channel *unique to the Dhan feed*, absent from FI-2010 and Kite.

These define named feature sets used for ablation: `base40` (raw, 40), `micro` (+6 scalars, 46), `orders60` (+order counts, 60), `depth80` (20 levels, 80), and `all` (66).

3.3.2 Dimensionality Reduction

None applied; the input is intentionally the raw/engineered LOB tensor. We instead study which *representation* the signal needs via the depth choice and the feature-set ablation below.

3.3.3 Feature Impact Observations

Table 5 shows the ablation for the proposed MambaLOB on NIFTY (improvement in test weighted-F_1 over `base40`). Findings: (i) **microstructure features help** — `micro` and `all` beat raw LOB, with the gain growing with horizon (at k=100 the `all` set lifts weighted-F_1 from 0.569 to 0.637); the effect also holds for TLOB (+0.007). (ii) **Raw order counts help only in combination** with the micro scalars (`orders60` alone hurts, but `all` is best) — a feature-interaction result that exploits the Dhan-unique channel. (iii) **Book depth beyond level 10 is uninformative** (`depth80` neutral) — the NSE index-futures signal lives at the top of book. (iv) The features add negligible parameters (68k → 70k), so MambaLOB stays the efficient model. The separate labelling-threshold study (Section 3.6) remains the key *economic*

Table 5: Feature ablation — MambaLOB (NIFTY), Δ weighted-F_1 vs `base40`.

Feature set	Δ (k=10)	Δ (k=100)	Verdict
`micro`	+0.028	+0.040	helps
`orders60`	−0.069	−0.007	hurts alone
`depth80`	−0.003	+0.002	≈ neutral
`all`	**+0.024**	**+0.068**	**best**

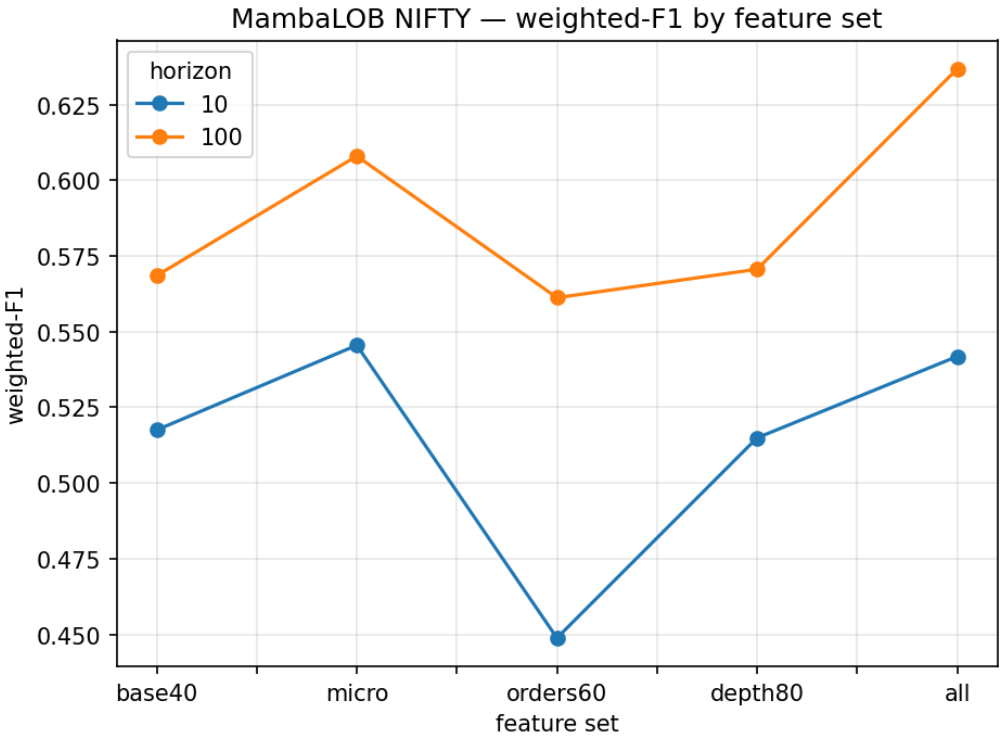

Figure 5: MambaLOB (NIFTY) weighted-F_1 by feature set and horizon. The engineered `micro` and `all` sets sit above raw `base40`, with the gain widening at the longer horizon; deep levels (`depth80`) and order counts alone (`orders60`) do not help in isolation.

finding. The multi-seed study (Section 3.6, Table 9) confirms this feature-engineering gain is statistically significant (p=0.001).

3.4 Technical Implementation

3.4.1 Tools and Technology Stack

Python; PyTorch with `mamba-ssm`/`causal-conv1d` (CUDA); scikit-learn; NumPy/Pandas; Dhan and Kite WebSocket APIs; AWS EC2 (collection) and S3 (storage, region `ap-south-2`); Google Colab and Kaggle (GPU training); `boto3`/`s3fs`; Matplotlib/Seaborn; Git/GitHub for version control.

3.4.2 System Setup and Environment Configuration (Data Engineering)

Because no clean Indian index-futures LOB dataset existed, the dataset was *engineered end to end*:

- **Reverse-engineering the Dhan binary feed**: the depth WebSocket emits an undocumented binary packet. We decoded its structure offline — a 1992-byte full format (40-byte header; 20×16-byte bid then ask records of order-count/price(f64)/qty) and a 664-byte compact format of two 332-byte sub-packets — fixing byte-offset bugs (bid price at +4, not +0) and rejecting denormalized near-zero floats from empty levels.
- **Collection pipeline**: an EC2 (Ubuntu) collector streams 20-level depth during market hours (09:15–15:30 IST), writes daily zstd Parquet, and a cron job syncs to S3 after close.
- **Production debugging**: corrected the futures segment code (`NSE_FNO`); diagnosed a mislabelled-instrument bug caused by a broker token reassignment after a 1:1 bonus issue; handled graceful WebSocket close at market end.
- **Contract roll**: the May contract expired and collection rolled to the June contract; the loader stitches them into a continuous front-month series with the roll as a segment boundary.
- **Training environments**: FI-2010 reproduction and Mamba/NSE training run on Colab/Kaggle GPUs; secrets (GitHub PAT, AWS keys, Dhan/Kite tokens) are read host-agnostically; every experiment uploads its results and checkpoints to S3 as it completes (timeout-safe, resumable across sessions).

3.4.3 Modular Code-base Description

The repository (`nse-lob-capstone`) is organised as: `data_acquisition/` (Dhan/Kite collectors, token refresh, S3 sync, EC2 ops); `modeling/` (`fi2010_dataset.py`, `nse_dataset.py`, `models.py` [DeepLOB/MLPLOB/MambaLOB], `tlob.py`, `train.py`, `backtest.py`, `stats.py`, `nbenv.py`); `notebooks/` (reproduction, TLOB, Mamba+efficiency, NSE matrix, NSE extras); `literature/`; and `docs/`. All models consume one data contract, so a single loader/trainer serves both datasets.

3.5 Baseline Model Development

3.5.1 Baseline Algorithm Description

DeepLOB (CNN–Inception–LSTM), MLPLOB (MLP+BiN), and TLOB (dual-attention transformer) are implemented to the shared $(B, 100, 40) \to [B, 3]$ contract; TLOB is vendored faithfully from the official implementation.

3.5.2 Training Setup and Parameters

Max 50 epochs (FI-2010) / 20 epochs (NSE), early stopping on validation F_1; AdamW; class-weighted cross-entropy; batch 128; per-model learning rates following the papers (DeepLOB/MLPLOB 10^{-3}, TLOB 10^{-4}, MambaLOB 3×10^{-4}).

3.5.3 Early Performance Metrics — FI-2010 reproduction

Table 6: FI-2010 (CF_7, $L = 100$) — weighted / macro F_1 (%).

Model	k=10	k=20	k=50	k=100	Notes
DeepLOB	82.5 / 71.9	—	—	—	≈ paper 83.4 (weighted)
TLOB	88.0 / 81.1	77.3 / 70.2	88.4 / 87.5	92.5 / 92.5	SOTA-range; paper ∼92.8 @ seq 128
MambaLOB (M1)	79.2 / 67.4	71.1 / 61.4	72.8 / 70.0	69.3 / 68.7	lower accuracy, far cheaper

Acceptance. DeepLOB's weighted F_1 (82.5%) matches the paper's headline (∼83.4%); TLOB is in the SOTA range, reaching 92.5% at k=100 (the paper's ∼92.8 was obtained at sequence length 128; we use 100). These confirm the reproduction is correct.

3.5.4 Output Visualization and Interpretation

We report confusion matrices per (model, horizon), F_1-by-horizon line charts, and the efficiency scaling curve (Section 3.10). Figure 6 shows the FI-2010 macro-F_1 trend across horizons; Figure 7 the DeepLOB confusion matrices at the shortest and longest horizons.

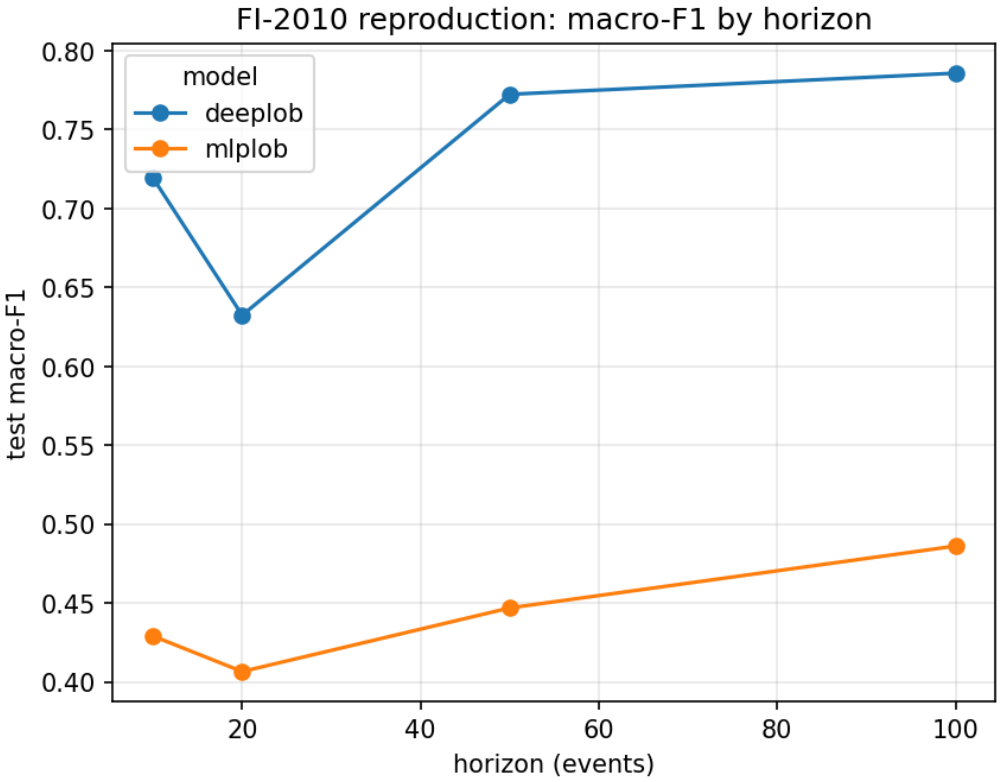

Figure 6: FI-2010 reproduction: macro-F_1 by prediction horizon for the reproduced baselines. The ordering and absolute range match the published DeepLOB/TLOB results and the independent LOBCAST reproduction.

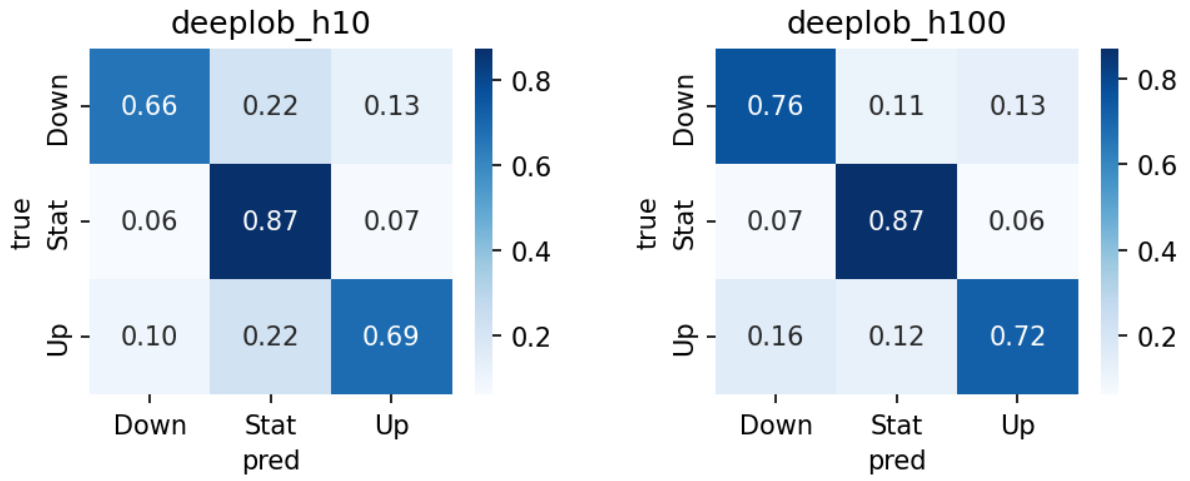

Figure 7: DeepLOB confusion matrices on FI-2010 at k=10 (left) and k=100 (right). At the longer horizon the Stationary class shrinks and predictions concentrate on the directional classes.

3.6 Preliminary Analysis and Observations

3.6.1 Result Analysis and Discussion

FI-2010. On accuracy, TLOB > DeepLOB > MambaLOB. **NSE transfer (Scheme A, complete — 32 runs over 4 models × 2 symbols × 4 horizons).** Table 7 summarises weighted F_1. All models beat the naive baselines (random ≈0.33; lift over the best baseline is positive everywhere), so there *is* learnable signal on NSE — but the absolute numbers are well below FI-2010, the expected degradation on new data (consistent with the LOBCAST benchmark study).

Table 7: NSE Scheme A — test weighted F_1 (range over $k \in \{10, 20, 50, 100\}$). Lift = weighted F_1 minus the best naive baseline.

Model	NIFTY (wF1)	BANKNIFTY (wF1)	Lift over baseline
DeepLOB	0.44–0.52	0.46–0.50	0.03–0.12
MLPLOB	0.46–0.64	0.35–0.47	0.01–0.19
MambaLOB	0.52–0.59	0.45–0.48	0.02–0.17
TLOB	**0.60–0.71**	**0.64–0.72**	**0.25–0.30**

TLOB leads across both instruments and all horizons (lift ≈0.25–0.30), mirroring its FI-2010 ranking. **Key transfer finding for the proposed model:** on NIFTY, *MambaLOB (68k parameters) outperforms DeepLOB (191k) and matches MLPLOB, second only to TLOB* — whereas on FI-2010 MambaLOB trailed DeepLOB. In other words, on the target Indian market the selective-SSM transfers *better* than on the benchmark, delivering competitive accuracy at roughly 1/27 of TLOB's parameters. On BANKNIFTY MambaLOB is on par with DeepLOB (both ≈0.46–0.48), below TLOB. Macro F_1 (NIFTY) follows the same order: DeepLOB 0.39–0.42, MLPLOB 0.44–0.47, MambaLOB 0.44–0.50, TLOB 0.53–0.58.

3.6.2 Error Profiling and Bias Detection

The dominant effect is class balance. Under the fixed threshold the Stationary class is small at long horizons (near-binary), whereas under the spread-relative scheme it dominates. Confusion matrices show errors concentrated on the minority directional classes, which is why macro F_1

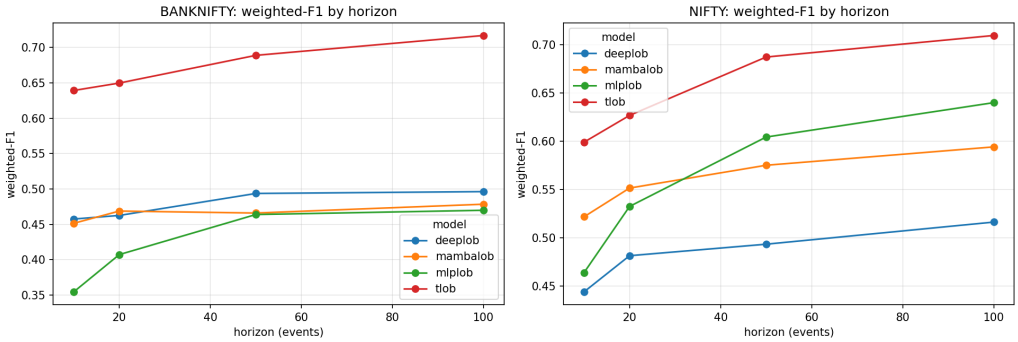

Figure 8: NSE transfer (Scheme A): test weighted-F_1 by horizon for each model. TLOB leads throughout; MambaLOB transfers competitively, matching or beating the CNN baseline on NIFTY at a fraction of the parameters.

trails weighted F_1.

3.6.3 Lessons Learned

(1) **Metric convention matters**: the FI-2010 headline (∼83.4) is a *weighted* F_1; our weighted F_1 matches it, while our macro F_1 (≈72) matches *independent* reproductions (LOBCAST). We therefore report both. (2) **F1 ≠ profit**: see Section 3.6 / 3.10. (3) **Mamba is primarily an efficiency win**: it trails on FI-2010 accuracy, yet on the NSE transfer it is competitive — beating the CNN baseline (DeepLOB) on NIFTY — at a fraction of the parameters and compute.

3.6.4 Improved MambaLOB Architectures (Tier-B)

We tested two architectural improvements to the vanilla SSM stack — *bidirectional* Mamba (a forward and a time-reversed scan, averaged) and *ConvMambaLOB* (DeepLOB's Conv+Inception spatial front-end feeding a Mamba temporal core, replacing DeepLOB's sequential LSTM) — plus their combination, all trained on the best (`all`) feature set against the TLOB transformer. Figure 9 contrasts the four variants; Table 8 reports the mean over both instruments and $k \in \{10, 100\}$.

Table 8: Tier-B architectures (NIFTY+BANKNIFTY, mean over $k \in \{10, 100\}$, `all` features, single seed).

Model	Params	wF1	mF1	% of TLOB wF1
MambaLOB (base)	**70k**	0.595	0.508	89%
+ bidirectional	135k	0.597	0.509	90%
+ Conv (ConvMambaLOB)	144k	**0.602**	**0.518**	**90%**
+ both	209k	0.587	0.508	88%
TLOB (SOTA)	1.80M	**0.666**	**0.570**	100%

The result is an efficiency trade-off, not an accuracy win. TLOB still leads weighted *and* macro F_1 in every (instrument, horizon) cell; our variants win none, trailing by ≈0.06–0.08 wF1. The contribution is the trade-off: **ConvMambaLOB recovers ∼90% of the transformer's quality with** 12.5× **fewer parameters**. Among our own designs the hybrid *spatial-convolution* front-end is the decisive choice (best or near-best in all four cells); bidirectionality is roughly neutral on average (it helps BANKNIFTY k=100 by +0.037 but hurts NIFTY k=10), and *stacking* both is the weakest Mamba variant — more parameters, no gain, consistent with

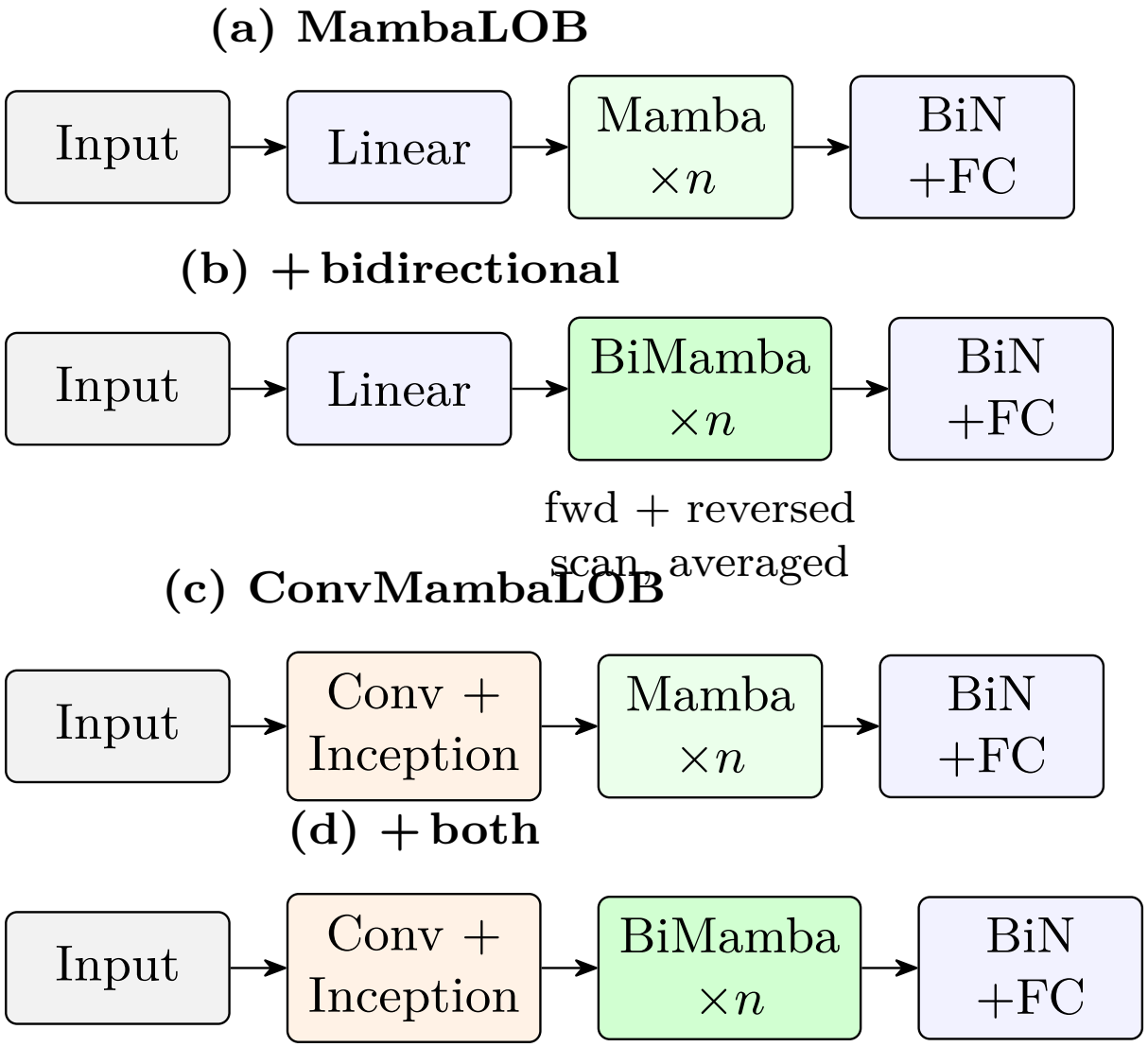

Figure 9: The four Tier-B variants. (a) base MambaLOB; (b) bidirectional Mamba (forward + time-reversed scan, averaged); (c) ConvMambaLOB (spatial conv front-end → Mamba core); (d) both combined. All share the Bilinear-Normalization head. Variant (c) was the best of the four; (d) the weakest (Table 8).

mild overfitting on the single-seed, one-month NSE sample. Multi-seed confidence intervals (Tier C, below) test which of these effects survive.

3.6.5 Statistical Significance and Calibration (Tier-C)

We re-ran the headline configurations over **3 seeds** (48 runs) and tested each claim two ways: a *paired* test across the 12 (seed, instrument, horizon) cells, and a *per-sample* paired bootstrap + McNemar test on a representative cell (NIFTY, k=100). Table 9 summarises.
(1) Microstructure feature engineering is significant by every test — the effect is large (across-seed +0.084 weighted-F_1, 11/12 cells) and especially dramatic on BANKNIFTY, where the raw `base40` input collapses to ≈0.45 while the engineered `all` set reaches ≈0.61. This is the strongest data-science result: the engineered features are what make the NSE signal learnable. **(2) The ConvMambaLOB gain is real in direction but not robust** — it wins 8/12 cells and is positive per-sample, but across seeds the +0.005 average lies within seed noise (p=0.25). The per-sample test flags it "significant" only because n=341,214 makes any tiny

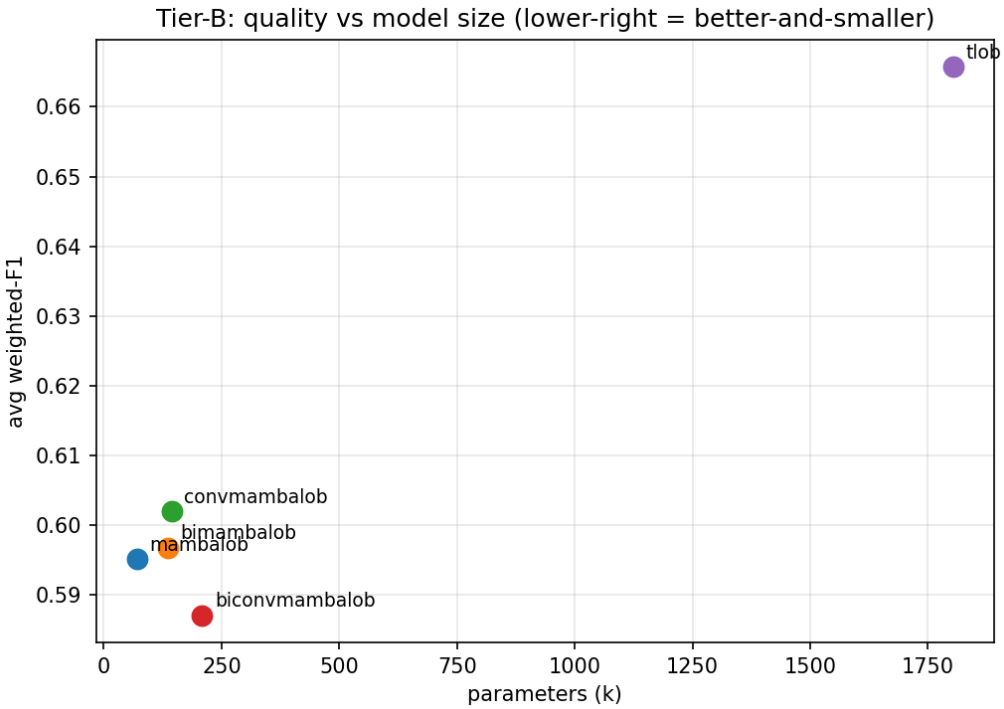

Figure 10: Quality vs. model size for the Tier-B architectures (average weighted-F_1 against parameter count). The Mamba family clusters at the low-parameter end near TLOB's accuracy; ConvMambaLOB is the best of the proposed variants. The lower-right region (better-and-smaller) is the design target.

Table 9: Tier-C significance (weighted-F_1). Across-seed: paired t-test over 12 cells. Per-sample: paired bootstrap 95% CI + McNemar at NIFTY k=100.

Claim	Across-seed Δ (p)	Per-sample Δ	Verdict
Features help (Mamba `all` vs `base40`)	+0.084 (p=0.001)	+0.043 (CI excl. 0)	**significant**
ConvMambaLOB vs base MambaLOB	+0.005 (p=0.25)	+0.015 (CI excl. 0)	not robust
Mamba / ConvMamba vs TLOB	−0.07 (p<0.001)	−0.087 (CI excl. 0)	TLOB ahead (sig.)

difference significant; we therefore report it honestly as a small, consistent-but-not-significant improvement. **(3) TLOB's accuracy lead is statistically robust** (p<0.001, 0/12 cells), confirming the contribution is the efficiency trade-off (competitive accuracy at 12–25× fewer parameters), not parity. **Calibration:** the proposed model is already well-calibrated (test ECE = 0.041; the NLL-optimal temperature is T=1.00, i.e. no scaling needed), so its predicted probabilities can be used directly in the cost-aware backtest. The methodological lesson — per-sample tests over-detect at large n, so the across-seed paired test is the meaningful measure of a *generalizable* improvement — is itself a reportable finding.

3.7 Originality and Innovation in Approach

3.7.1 Creative Framing of the Problem

The first ML-ready NSE index-futures LOB benchmark, framed honestly for snapshot depth data (an "event" is a change in the top-of-book), and a transaction-cost-relative labelling scheme that exposes when signal is economically meaningful.

3.7.2 Experimental Innovations

MambaLOB (selective SSM for LOB classification); **ConvMambaLOB**, a hybrid that fuses DeepLOB's spatial convolutions with a linear-time selective-SSM temporal core (Section 3.6); a controlled long-context experiment exploiting Mamba's linear scaling; and two labelling schemes (fixed vs spread-relative) reported side by side.

3.8 Risk Assessment and Project Management

3.8.1 Progress Tracker vs. Original Timeline

Completed: literature survey; the engineered NSE dataset + pipeline; FI-2010 reproduction (DeepLOB, MLPLOB, TLOB); MambaLOB on FI-2010 + efficiency analysis; the *complete* NSE Scheme-A matrix (all four models, both instruments, four horizons); the Phase-2 microstructure feature ablation (Tier-A), the improved architectures (Tier-B), and the multi-seed significance + calibration study (Tier-C). Deferred (next phase): hyperparameter optimisation and the class-imbalance study (both implemented as a dual-GPU, S3-resumable notebook, pending a final run), Scheme-B confirmatory training, and the cost-aware backtest.

3.8.2 Bottlenecks and Delays

Free-tier GPU throttling (mitigated by moving from Colab to Kaggle, which has a separate quota); a Kaggle P100 incompatible with the latest PyTorch CUDA build (use T4); `mamba-ssm` failing to build under pip build isolation; and session time-outs on long matrices.

3.8.3 Risk Mitigation Strategies

`mamba-ssm` installed with `--no-build-isolation`; per-run S3 upload + pull-on-start makes every long matrix resumable (a time-out costs at most the run in flight); host-agnostic secrets/data so the same notebooks run on Colab or Kaggle.

3.9 Experimental Reproducibility

3.9.1 Data Splits and Control Measures

FI-2010: standard CF_7 (7 train / 1 val / 3 test days). NSE: chronological front-month split (train 12 May–4 June; test 5–12 June). Normalization statistics are computed on training data only; windows and labels never cross a day/contract boundary.

3.9.2 Random Seed Control

A `set_seed` utility seeds Python/NumPy/PyTorch; the headline configurations were run over 3 seeds with across-seed 95% confidence intervals and paired significance tests (Section 3.6, Table 9). All results, metrics JSON, and checkpoints are versioned on S3 under `experiments/`.

3.10 Scalability and Efficiency Considerations

3.10.1 Memory, Time, and Resource Profiling

Headline efficiency result. MambaLOB's parameter count is constant in L and its memory/latency grow *linearly*, while TLOB grows *quadratically*. At L=800 MambaLOB is $\approx$31$\times$ faster, uses $\approx$3.4$\times$ less memory, and has $\approx$750$\times$ fewer parameters — a clean demonstration of the linear-scaling advantage that motivates the architecture. (FLOP counts for Mamba are under-reported by the profiler because the fused CUDA kernel is not traced; we therefore rely on measured latency and memory.)

Table 10: Efficiency at L=100, batch 1 (inference).

Model	Params	Latency (ms)	Peak mem (MB)
DeepLOB	191k	1.5	30
MLPLOB	529k	0.2	22
TLOB	1.80M	6.1	27
MambaLOB	**68k**	1.5	20

Table 11: Scaling with sequence length (batch 32): TLOB $O(L^2)$ vs MambaLOB $O(L)$.

L	TLOB params	TLOB mem / latency	Mamba params	Mamba mem / latency
100	1.8M	49 MB / 11 ms	68k	32 MB / 1.5 ms
400	13.5M	164 MB / 54 ms	68k	69 MB / 1.8 ms
800	51M	400 MB / 164 ms	68k	118 MB / 5.3 ms

3.10.2 Scalability Constraints

Training throughput is bounded by data loading and the sequential scan rather than raw FLOPs; the spread-relative task is degenerate at short horizons; and real-time deployment is out of scope.

3.11 Ethical Considerations and Responsible AI Use

3.11.1 Generative AI Usage Disclosure

Generative-AI assistance (a coding/writing assistant) was used for code scaffolding, debugging, drafting documentation, and report formatting. All generated content and code were reviewed, tested, and validated before inclusion.

3.11.2 Bias and Fairness in Dataset

The data are public benchmark data and self-collected public market data; there is no personal or demographic data. The main "bias" risk is class imbalance, which we address with class-weighted loss and by reporting macro F_1 and baselines.

3.12 Feedback Incorporation and Continuous Improvement

3.12.1 Peer and Mentor Feedback Summary

Feedback drove: separating weighted vs macro F_1 reporting; checking the spread-relative label balance before training (which revealed degeneracy and saved a large, pointless run); and prioritising the data-engineering narrative.

3.12.2 Reflection and Learning Journey

The project built skills across data engineering (binary protocol decoding, cloud collection, robust preprocessing), reproducible deep-learning experimentation, and honest empirical evaluation under cost and compute constraints.

3.13 Chapter Summary and Next Steps

We engineered an NSE index-futures LOB dataset, reproduced three baselines on FI-2010, introduced MambaLOB with a strong efficiency result, and completed the NSE Scheme-A transfer study (where MambaLOB transfers competitively, beating the CNN baseline on NIFTY).

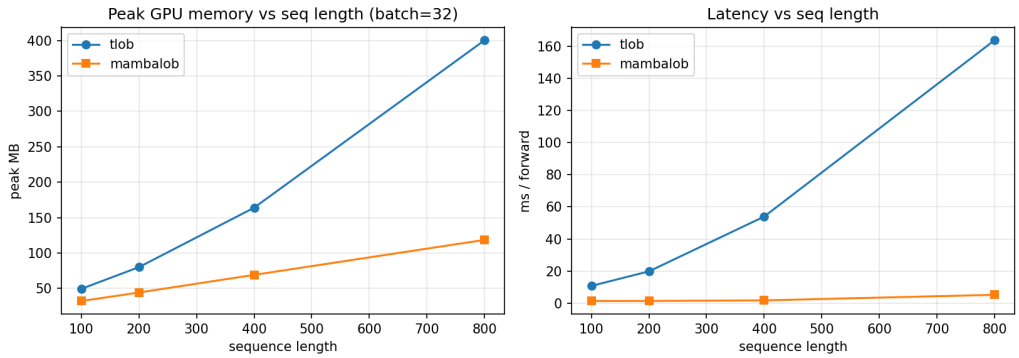

Figure 11: Sequence-length scaling: TLOB's $O(L^2)$ attention versus MambaLOB's $O(L)$ selective scan. The gap widens sharply with L, the basis of the architecture's efficiency claim.

Next steps — Phase 2 (planned, pending mentor discussion). The work so far is strong on data engineering, reproduction, and transfer; Phase 2 deepens the *Data-Science* contribution so the novelty is unambiguous. It has three tiers:
*(A) Microstructure feature engineering (DE→DS bridge; novel) — **done**, see Section 3.3.* Engineered OFI, depth imbalance, micro-price, relative spread, and the Dhan-unique order-count channel, ablated against the raw window. Result: microstructure features help (best set lifts MambaLOB weighted-F_1 by up to +0.068 at k=100), order counts help only in combination, and book depth beyond level 10 is uninformative. Multi-seed CIs (Tier C) remain to confirm significance.
*(B) Improved MambaLOB architecture (the headline novelty) — **done**, see Section 3.6.* Built and evaluated (i) **bidirectional Mamba** and (ii) **ConvMambaLOB** (DeepLOB convolutions → linear-time Mamba core), plus their combination, against TLOB. Result: ConvMambaLOB recovers ∼90% of TLOB's weighted-F_1 at 12.5× fewer parameters — an efficiency win, not an accuracy win (TLOB still leads). The spatial-convolution front-end is the decisive design; bidirectionality and stacking give diminishing returns. Multi-seed CIs (Tier C) remain to confirm significance.
*(C) Rigour layer (expected practice) — **significance + calibration done**, see Section 3.6, Table 9.* The 3-seed study with paired across-seed and per-sample tests confirmed the feature-engineering gain (p=0.001) and the (significant) gap to TLOB, showed the ConvMambaLOB gain is not robust across seeds, and found the proposed model already well-calibrated (ECE 0.041, T=1.00). Remaining: hyperparameter optimization (Optuna, equal budget for baselines) and the class-imbalance study (focal vs class-weighted CE vs balanced sampling vs label smoothing) — both implemented as a dual-GPU, S3-resumable notebook and pending a final run.
Also carried over from the interim plan: the Scheme-B confirmatory training, the cost-aware backtest, a walk-forward robustness check, and consolidation into the final report. A standalone `docs/phase2_plan.md` tracks this in detail.

Appendices (planned). Detailed per-(model, symbol, horizon) result tables; confusion-matrix and scaling figures; the notebook→result map; and reproducibility artefacts (`requirements`, seeds, S3 paths).